Ministry of Education of Republic of Moldova

Technical University of Moldova

Faculty of Computers, Informatics and Microelectronics

Software Engineering Department

Embedded Systems

Laboratory work work #4.1

Actuator Control and Signal Conversion

During the preparation of this report, the author used various AI Tools such as ChatGPT, Gemini, Claude to generate/augment the content, generate the code and structure the directory. The resulting information was reviewed, validated, and adjusted to meet the requirements of the laboratory assignment.

Author: Timur Cravtov
std. gr. FAF-231

Verified by:
Alexei Martîniuc
asist. univ

Chișinău, 2026

1 Objective and Technical Analysis

The objective of this laboratory work is to implement a servo motor control system with multi-stage signal conditioning for embedded systems. This involves controlling a servo motor's angular position (0–180°) through a conditioning pipeline that includes saturation, median filtering, weighted averaging, and ramping for actuator protection. The system implements a three-stage control pipeline: **Input Processing**, **Signal Conditioning**, and **Servo Actuator Control with Feedback**.

Through this work, the following learning outcomes are pursued:

- Understanding servo motor control principles and PWM signaling.
- Implementing multi-stage signal conditioning: saturation, median filter, weighted average, and ramping.
- Controlling servo motors with precise angular positioning (0–180°).
- Managing real-time input/output through multiple interfaces (keypad, LCD).
- Designing modular software architecture using FreeRTOS with dedicated tasks.
- Optimizing embedded code using PROGMEM and printf_P for memory efficiency.

1.1 Servo Motor Control

1.1.1 Servo Fundamentals

Servo motors use PWM signals for position control. Standard hobby servos expect a PWM pulse every 20ms, with pulse widths ranging from 1ms (0°) to 2ms (180°).

The relationship between pulse width and angle:

$$\theta = \frac{t_{pulse} - 1ms}{1ms} \times 180 \tag{1}$$

The Arduino Servo library abstracts this, allowing direct angle specification (0–180°). In this implementation, the user controls position as a percentage (0–100%), which maps to the servo angle.

1.2 Signal Conditioning Pipeline

The signal conditioning pipeline applies multiple processing stages to ensure safe and smooth actuator control.

1.2.1 Saturation

Saturation clamps the input to valid physical limits:

$$y = \begin{cases} 0 & x < 0 \\ x & 0 \leq x \leq 100 \\ 100 & x > 100 \end{cases} \tag{2}$$

1.2.2 Median Filter

A median filter with window size 5 removes impulse noise (outliers). It sorts the last 5 samples and outputs the middle value, making it robust against sporadic erroneous readings.

1.2.3 Weighted Average

A weighted average with coefficients [5, 3, 2] reduces fluctuations while prioritizing recent values:

$$y = \frac{5 \cdot x[n] + 3 \cdot x[n-1] + 2 \cdot x[n-2]}{10} \tag{3}$$

1.2.4 Ramping

Ramping limits the rate of change to protect the actuator from abrupt movements:

$$y[n] = \begin{cases} y[n-1] + \Delta_{max} & x[n] - y[n-1] > \Delta_{max} \\ y[n-1] - \Delta_{max} & x[n] - y[n-1] < -\Delta_{max} \\ x[n] & \text{otherwise} \end{cases} \tag{4}$$

With $\Delta_{max} = 5$ per cycle, the servo smoothly transitions to the target position over multiple cycles.

1.3 Memory Optimization

For AVR microcontrollers with limited RAM, string literals should be stored in flash memory using PROGMEM:

- `PROGMEM` keyword stores constants in flash (not RAM)
- `printf_P()` prints from PROGMEM strings
- `PSTR()` creates inline PROGMEM strings

2 System Design

2.1 System Architecture

The system architecture follows a modular task-based design using FreeRTOS. Three tasks handle specific responsibilities:

- **TaskRead** (50ms): Reads user input from keypad, validates, and updates raw position.
- **TaskFilter** (100ms): Applies signal conditioning pipeline and controls servo.
- **TaskDisplay** (500ms): Updates LCD with current status.

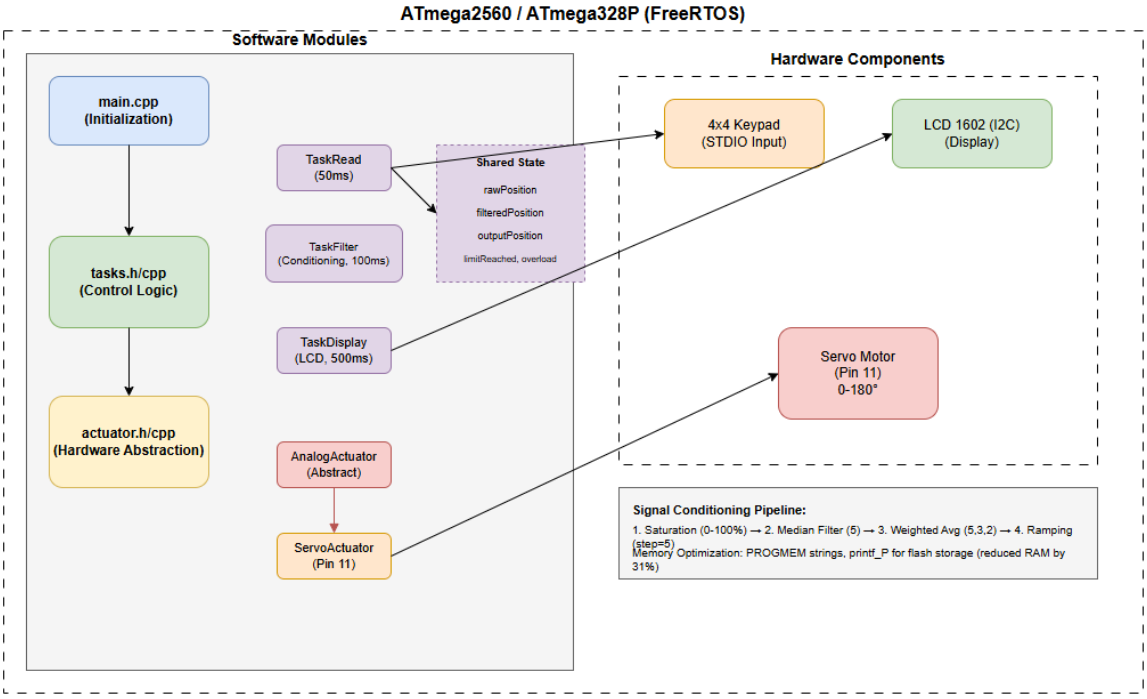

Figure 1: System Architecture Diagram for Lab 4.2

2.2 Circuit Diagram

The hardware connections for the servo control system include:

2.2.1 Servo Motor (SG90) Connections

- **VCC (Red)**: Connected to 5V power supply
- **GND (Brown)**: Connected to ground rail
- **Signal (Orange)**: Connected to digital pin D11
- **Position Mapping**: 0% duty = 0°, 100% duty = 180°

2.2.2 LCD 1602 (I2C) Connections

- **VCC**: Connected to 5V
- **GND**: Connected to ground
- **SDA**: Connected to pin A4 (Nano) or D20 (Mega)
- **SCL**: Connected to pin A5 (Nano) or D21 (Mega)
- **I2C Address**: 0x27

2.2.3 4x4 Keypad Connections

- **Row Pins (R1-R4)**: Connected to D9, D8, D7, D6
- **Column Pins (C1-C4)**: Connected to D5, D4, D3, D2

2.2.4 Pin Assignment Summary

Table 1 summarizes the complete pin assignments.

Table 1: Microcontroller Pin Assignments

Component	**Pin**	**Function**
Servo Signal	D11	PWM output
LCD SDA	A4 (Nano) / D20 (Mega)	I2C data
LCD SCL	A5 (Nano) / D21 (Mega)	I2C clock
Keypad R1	D9	Row 1
Keypad R2	D8	Row 2
Keypad R3	D7	Row 3
Keypad R4	D6	Row 4
Keypad C1	D5	Column 1
Keypad C2	D4	Column 2
Keypad C3	D3	Column 3
Keypad C4	D2	Column 4

2.3 Signal Conditioning Pipeline

The signal conditioning applies four stages in sequence: saturation → median filter → weighted average → ramping.

3 Implementation and Results

3.1 Software Architecture

3.1.1 PROGMEM String Storage

String literals are stored in flash memory to conserve RAM:

```
static const char STR_LINE0[] PROGMEM = "\rOut:%3d%% A:%3d\n";
static const char STR_OK[] PROGMEM = "\rStatus: OK      ";
static const char STR_LIMIT[] PROGMEM = "\r!LIMIT REACHED!";
static const char STR_OVERLOAD[] PROGMEM = "\r!OVERLOAD!     ";
```

3.1.2 ServoActuator Implementation

```
class ServoActuator : public AnalogActuator {
  private:
    uint8_t pin;
    Servo servo;
    uint8_t currentPosition;
    uint8_t currentAngle;
  public:
    void setPosition(uint8_t position) override {
        if (position > 100) position = 100;
        currentPosition = position;
        currentAngle = map(position, 0, 100, 0, 180);
        servo.write(currentAngle);
    }
};
```

3.1.3 Signal Conditioning

```
// 1. Saturation
uint8_t saturated = (rawPosition > 100) ? 100 : rawPosition;

// 2. Median filter (window=5)
medianBuffer[medianIndex] = saturated;
medianIndex = (medianIndex + 1) % 5;
uint8_t sorted[5];
for (uint8_t i = 0; i < 5; i++) sorted[i] = medianBuffer[i];
sortArray(sorted, 5);
uint8_t medianValue = sorted[2];

// 3. Weighted average (5,3,2)
filteredPosition = (historyBuffer[0] * 5
                  + historyBuffer[1] * 3
                  + historyBuffer[2] * 2) / 10;

// 4. Ramping (step=5)
int8_t diff = filteredPosition - outputPosition;
if (diff > 5) outputPosition += 5;
else if (diff < -5) outputPosition -= 5;
else outputPosition = filteredPosition;
```

3.1.4 LCD Display (Static 2-Line)

```
void TaskDisplay(void* pvParameters) {
    while (true) {
        // Line 0: Output and angle
        printf_P(STR_LINE0, sysState.output, sysState.angle);

        // Line 1: Status
        if (sysState.overloadAlert)
            printf_P(STR_OVERLOAD);
        else if (sysState.limitAlert)
```

```
            printf_P(STR_LIMIT);
        else
            printf_P(STR_OK);

        printf_P(PSTR("\n"));  // Back to line 0
        vTaskDelay(pdMS_TO_TICKS(500));
    }
}
```

3.2 Keypad Control Mapping

Table 2 shows the keypad key assignments.

Table 2: Keypad Key Assignments

Key	**Function**
0-9	Enter position digits (0-100)
# or A	Confirm input
* or B	Cancel input
C	Emergency stop (0%)
D	Full position (100%)

3.3 Memory Usage

Table 3: Memory Usage (Arduino Nano)

Metric	**Before Optimization**	**After Optimization**
RAM	1149 bytes (56%)	794 bytes (39%)
Flash	19686 bytes (64%)	17744 bytes (58%)

3.4 Performance Characteristics

Table 4: System Performance Metrics

Metric	**Value**
TaskRead period	50ms
TaskFilter period	100ms
TaskDisplay period	500ms
Median filter window	5 samples
Weighted average coefficients	[5, 3, 2]
Ramping step	5% per cycle
Servo range	0–180°

3.5 Simulation Results

The system was validated using Wokwi simulation platform for both Arduino Mega and Nano targets.

4 Conclusion

This laboratory work successfully implemented a servo motor control system with multi-stage signal conditioning. The conditioning pipeline—saturation, median filter, weighted average, and ramping—ensures safe and smooth actuator operation.

Key achievements include:

- Multi-stage signal conditioning for actuator protection
- Memory optimization using PROGMEM (reduced RAM by 31%)
- Static 2-line LCD display with efficient updates
- Multi-digit keypad input with validation
- FreeRTOS task-based architecture

The median filter effectively removes impulse noise, while the weighted average smooths fluctuations. Ramping prevents mechanical stress on the servo motor during rapid position changes.

Future enhancements could include:

- Configurable conditioning parameters
- Multiple servo support
- Position feedback integration
- PID control for precise positioning

References

[1] FreeRTOS API Reference, `https://www.freertos.org/a00106.html`

[2] Hobby Servo Fundamentals, Princeton University

[3] Pitas, I., Venetsanopoulos, A.N., "Nonlinear Digital Filters", Springer, 1990.

[4] Barr, M., Massa, A., "Programming Embedded Systems in C and C++", O'Reilly Media, 2006.

[5] Arduino Servo Library Reference, https://www.arduino.cc/reference/en/libraries/servo/

A Source Code

Full source code for Lab 4.2 is available in the repository.

actuator.h

```
#pragma once

#include <Arduino.h>
#include <Servo.h>

// Configuration constants
constexpr uint8_t ACTUATOR_MIN = 0;
constexpr uint8_t ACTUATOR_MAX = 100;

// Abstract interface for analog actuators
class AnalogActuator {
  public:
    virtual void setPosition(uint8_t position) = 0;
    virtual uint8_t getPosition() = 0;
    virtual void begin() = 0;
    virtual ~AnalogActuator() {}
};

// Concrete implementation for Servo-based actuator control
// Maps 0-100% to servo angle (0-180 degrees)
class ServoActuator : public AnalogActuator {
  private:
    uint8_t pin;
    Servo servo;
    uint8_t currentPosition;  // 0-100%
    uint8_t currentAngle;     // 0-180 degrees

  public:
    ServoActuator(uint8_t p);
    void begin() override;
    void setPosition(uint8_t position) override;
```

```
    uint8_t getPosition() override;
    uint8_t getAngle();
};
```

Listing 1: actuator.h - Actuator Interface

actuator.cpp

```
#include "actuator.h"

ServoActuator::ServoActuator(uint8_t p) : pin(p),
    currentPosition(0), currentAngle(0) {}

void ServoActuator::begin() {
    servo.attach(pin);
    servo.write(0);  // Start at 0 degrees
}

void ServoActuator::setPosition(uint8_t position) {
    // Saturate to valid range
    if (position > ACTUATOR_MAX) position = ACTUATOR_MAX;

    currentPosition = position;
    // Map 0-100% to 0-180 degrees
    currentAngle = map(position, ACTUATOR_MIN, ACTUATOR_MAX, 0,
        180);
    servo.write(currentAngle);
}

uint8_t ServoActuator::getPosition() {
    return currentPosition;
}

uint8_t ServoActuator::getAngle() {
    return currentAngle;
}
```

Listing 2: actuator.cpp - Actuator Implementation

tasks.h

```
#ifndef TASKS_H
```

```
#define TASKS_H

#include "actuator.h"
#include <Arduino_FreeRTOS.h>

// Task periods (ms)
constexpr uint32_t TASK_READ_PERIOD = 50;        // Keypad reading period
constexpr uint32_t TASK_FILTER_PERIOD = 100;     // Filtering period
constexpr uint32_t TASK_DISPLAY_PERIOD = 500;    // LCD display period

// Shared state variables (accessed by tasks)
extern volatile uint8_t rawPosition;         // Raw input position (0-100%)
extern volatile uint8_t filteredPosition;    // Filtered position (0-100%)
extern volatile uint8_t outputPosition;      // Output position with ramping (0-100%)
extern volatile uint8_t outputAngle;         // Servo angle (0-180 degrees)
extern volatile bool commandReady;           // Flag for new command
extern volatile bool limitReached;           // Alert: limit reached
extern volatile bool overloadDetected;       // Alert: overload

// System state
struct SystemState {
    uint8_t raw;
    uint8_t filtered;
    uint8_t output;
    uint8_t angle;
    bool limitAlert;
    bool overloadAlert;
};

```

```
extern volatile SystemState sysState;

// Task function declarations
void TaskRead(void* pvParameters);        // Read commands from keypad
void TaskFilter(void* pvParameters);      // Filter and condition signal
void TaskDisplay(void* pvParameters);     // Display on LCD

#endif // TASKS_H
```

Listing 3: tasks.h - Task Interface

tasks.cpp

```
#include "tasks.h"
#include <Arduino.h>
#include <Arduino_FreeRTOS.h>
#include <serialio/serialio.h>
#include <lcd/LcdStdioManager.h>
#include <avr/pgmspace.h>

// PROGMEM string literals to save RAM (for 16x2 LCD)
// \r returns to start of line, \n moves to next line
static const char STR_LINE0[] PROGMEM = "\rOut:%3d%% A:%3d\n"; // \r for overwrite, \n to go to line 1
static const char STR_OK[] PROGMEM = "\rStatus: OK        ";
static const char STR_LIMIT[] PROGMEM = "\r!LIMIT REACHED!";
static const char STR_OVERLOAD[] PROGMEM = "\r!OVERLOAD!       ";

// Shared state variables
volatile uint8_t rawPosition = 0;
volatile uint8_t filteredPosition = 0;
volatile uint8_t outputPosition = 0;
volatile uint8_t outputAngle = 0;
volatile bool commandReady = false;
volatile bool limitReached = false;
volatile bool overloadDetected = false;
volatile SystemState sysState = {0, 0, 0, 0, false, false};

// Median filter (window=5)
```

```c
static uint8_t medianBuffer[5];
static uint8_t medianIndex = 0;

// Weighted averaging (weights 5,3,2 sum=10)
static uint8_t historyBuffer[3] = {0, 0, 0};

// Input buffer
static char inputBuffer[4] = {0};
static uint8_t inputIndex = 0;

// Insertion sort for small arrays (more efficient than bubble)
static void sortArray(uint8_t* arr, uint8_t size) {
    for (uint8_t i = 1; i < size; i++) {
        uint8_t key = arr[i];
        int8_t j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

// Task 1: Read commands from keypad
void TaskRead(void* pvParameters) {
    (void)pvParameters;
    char c;

    while (true) {
        c = 0;
        if (scanf("%c", &c) > 0 && c > '\r') {
            if (c >= '0' && c <= '9') {
                if (inputIndex < 3) {
                    inputBuffer[inputIndex++] = c;
                    inputBuffer[inputIndex] = '\0';
                }
            } else if (c == '#' || c == 'A') {
                if (inputIndex > 0) {
```

```
                    uint8_t value = (uint8_t)atoi(inputBuffer);
                    if (value > 100) { value = 100; limitReached = true; }
                    else { limitReached = false; }
                    rawPosition = value;
                    commandReady = true;
                    inputIndex = 0;
                    inputBuffer[0] = '\0';
                }
            } else if (c == '*' || c == 'B') {
                inputIndex = 0;
                inputBuffer[0] = '\0';
            } else if (c == 'C') {
                rawPosition = 0;
                commandReady = true;
                overloadDetected = true;
                inputIndex = 0;
                inputBuffer[0] = '\0';
            } else if (c == 'D') {
                rawPosition = 100;
                commandReady = true;
                limitReached = true;
                inputIndex = 0;
                inputBuffer[0] = '\0';
            }
        }
        vTaskDelay(pdMS_TO_TICKS(TASK_READ_PERIOD));
    }
}

// Task 2: Filter and condition signal
void TaskFilter(void* pvParameters) {
    ServoActuator* actuator = static_cast<ServoActuator*>(pvParameters);

    // Initialize buffers
    for (uint8_t i = 0; i < 5; i++) medianBuffer[i] = 0;
    historyBuffer[0] = historyBuffer[1] = historyBuffer[2] = 0;
```

```

    while (true) {
        // 1. Saturation
        uint8_t saturated = (rawPosition > 100) ? 100 : rawPosition;

        // 2. Median filter
        medianBuffer[medianIndex] = saturated;
        medianIndex = (medianIndex + 1) % 5;

        uint8_t sorted[5];
        for (uint8_t i = 0; i < 5; i++) sorted[i] = medianBuffer[i];
        sortArray(sorted, 5);
        uint8_t medianValue = sorted[2];

        // 3. Weighted average (weights: 5,3,2)
        historyBuffer[2] = historyBuffer[1];
        historyBuffer[1] = historyBuffer[0];
        historyBuffer[0] = medianValue;
        filteredPosition = (historyBuffer[0] * 5 + historyBuffer[1] * 3 + historyBuffer[2] * 2) / 10;

        // 4. Ramping (step=5)
        int8_t diff = (int8_t)filteredPosition - (int8_t)outputPosition;
        if (diff > 5) outputPosition += 5;
        else if (diff < -5) outputPosition -= 5;
        else outputPosition = filteredPosition;

        // 5. Apply to actuator
        actuator->setPosition(outputPosition);
        outputAngle = actuator->getAngle();

        // Update state
        sysState.raw = rawPosition;
        sysState.filtered = filteredPosition;
        sysState.output = outputPosition;
```

```
        sysState.angle = outputAngle;
        sysState.limitAlert = limitReached;
        sysState.overloadAlert = overloadDetected;

        vTaskDelay(pdMS_TO_TICKS(TASK_FILTER_PERIOD));
    }
}

// Task 3: Display on LCD (16x2, static lines)
// LCD driver: \n toggles row (0<->1), \r goes to column 0 of current row
void TaskDisplay(void* pvParameters) {
    (void)pvParameters;

    while (true) {
        // Ensure we start on line 0 (toggle if on line 1)
        // Print line 0 with \n to go to line 1
        printf_P(STR_LINE0, sysState.output, sysState.angle);

        // Now on line 1 - print status/alert
        if (sysState.overloadAlert) {
            printf_P(STR_OVERLOAD);
            overloadDetected = false;
        } else if (sysState.limitAlert) {
            printf_P(STR_LIMIT);
        } else {
            printf_P(STR_OK);
        }

        // Print \n to go back to line 0 for next cycle
        printf_P(PSTR("\n"));

        vTaskDelay(pdMS_TO_TICKS(TASK_DISPLAY_PERIOD));
    }
}
```

Listing 4: tasks.cpp - Task Implementation

`main.cpp`

```
#include <Arduino.h>
```

```
#include <Arduino_FreeRTOS.h>
#include <Keypad.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <lcd/LcdStdioManager.h>
#include <keypad/KeypadStdioManager.h>
#include "actuator.h"
#include <serialio/serialio.h>
#include "tasks.h"

// LCD (I2C address 0x27, 16x2)
static LiquidCrystal_I2C lcd(0x27, 16, 2);

// Keypad pin configuration
static byte rowPins[4] = {9, 8, 7, 6};
static byte colPins[4] = {5, 4, 3, 2};

// Keypad layout (PROGMEM via Keypad library)
static const char keys[4][4] = {
    {'1', '2', '3', 'A'},
    {'4', '5', '6', 'B'},
    {'7', '8', '9', 'C'},
    {'*', '0', '#', 'D'}
};

static Keypad keypad(makeKeymap(keys), rowPins, colPins, 4, 4);

// Servo actuator on pin 11
static ServoActuator actuator(11);

void setup() {
    Serial.begin(9600);
    actuator.begin();

    lcd.init();
    lcd.backlight();

    // STDIO: LCD output, Keypad input
```

```
    static LcdStdioManager lcdManager;
    lcdManager.setup(&lcd);
    KeypadStdioManager::setup(&keypad);

    // redirectSerialToStdio(false, false, true);

    // FreeRTOS tasks (stack sizes tuned for Nano - 2KB RAM)
    xTaskCreate(TaskRead,    "Read",   100, NULL,       1, NULL);
    xTaskCreate(TaskFilter,  "Filter", 120, &actuator, 2, NULL);
    xTaskCreate(TaskDisplay, "Disp",   140, NULL,       1, NULL);

    // fprintf_P(stderr, PSTR("Ready\n"));
    vTaskStartScheduler();
}

void loop() {}
```

Listing 5: main.cpp - System Initialization